

EmbedGit

What we are building, how change detection works, and what must be versioned

CORE IDEA	SAFE REUSE	PROVENANCE
Track changes in source documents and deterministic chunks.	Regenerate only new, modified, or stale embeddings.	Record the exact pipeline that produced every snapshot.

A practical guide for the semester MVP

Prepared 08 August 2026

01

The project in one sentence

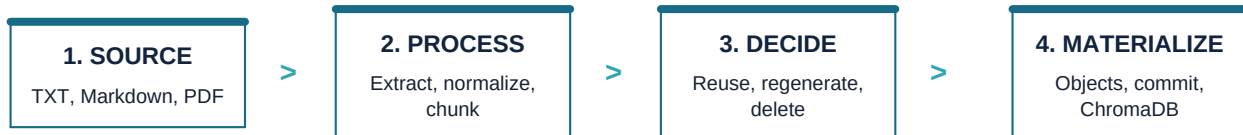
EmbedGit is a lifecycle manager for embedding collections. It tracks what changed, decides what can be reused, invokes an existing embedding model only where needed, and stores reproducible snapshots.

The main contribution is not a new AI model. It is the decision and versioning layer around an existing model.

The problem

A semantic-search or RAG system normally extracts documents, divides them into chunks, generates a vector for each chunk, and stores those vectors in a database. After a small edit, a naive pipeline may regenerate the entire collection. EmbedGit determines exactly which outputs remain valid.

End-to-end responsibility



What the software does

- Loads and normalizes documents.
- Creates deterministic chunks.
- Compares working state with the previous commit.
- Chooses reusable versus regeneration-required chunks.
- Stores vectors with provenance and synchronizes ChromaDB.

- Does not train a neural network.
- Does not implement vector mathematics from scratch.
- Does not infer document edits primarily from vector differences.
- Does not make different models automatically compatible.
- Does not need cloud or multi-user features for the MVP.

02

Identity, content and provenance

A single hash cannot answer every question. The design needs separate identifiers for logical history, exact content, generated objects, and the pipeline configuration.

Field	What it identifies	Primary use
document_id	The logical source document across commits	Scope comparisons and preserve history even if content changes
chunk_lineage_id	A logical chunk or its ancestry	Explain modified relationships; never sufficient for reuse
chunk_hash	Exact normalized chunk content plus relevant chunking context	Prove identical chunk content
config_hash	Effective extraction, preprocessing, chunking and model settings	Detect stale outputs and support reproduction
object_hash	The immutable stored vector object and its provenance	Deduplicate storage and verify integrity
source_span	Line, paragraph, page or character range	Map document edits to affected chunks

A stable chunk ID helps describe lineage. Only content and configuration hashes can authorize embedding reuse.

Example

Commit	Lineage ID	Chunk text	Chunk hash	Classification
C1	doc1:c2	A process is a running program.	bbb	Previous state
C2	doc1:c2	A process is a program being executed.	xyz	MODIFIED
C2	doc1:c3	Threads share process resources.	new	ADDED

Why the content hash alone is insufficient

A new hash only proves that the exact current content was not matched. It does not tell whether the item is a modification or a delete-plus-add. That classification requires document context, ordering, source spans, and optionally text similarity.

03

Git-like source diff plus chunk matching

Git does not assign permanent IDs to individual lines. It stores versions of file content and computes a diff. EmbedGit can use the same idea to locate edits before validating reuse at chunk level.

Source-level example

PREVIOUS	CURRENT
1 Introduction	1 Introduction
2 OS manages hardware.	2 OS manages hardware.
3 Processes execute programs.	3 Threads run inside processes. [ADDED]
4 Virtual memory manages RAM.	4 Processes execute programs.
	5 Virtual memory manages RAM.

Recommended hybrid algorithm

#	Operation	Purpose
1	Normalize and snapshot source	Create a comparable representation of old and current text.
2	Compute source diff	Locate inserted, deleted and replaced ranges by line, paragraph or character.
3	Map ranges to chunks	Mark intersecting chunks plus a small boundary window as candidates.
4	Rechunk affected regions	Apply the current deterministic chunking pipeline.
5	Exact-hash match first	Any identical chunk under the same configuration is reusable.
6	Pair unmatched neighbors	Use order, spans and optional text similarity to report MODIFIED.
7	Classify leftovers	Unmatched current chunks are ADDED; unmatched old chunks are DELETED.
8	Generate only when required	Embed ADDED, MODIFIED and STALE chunks; reuse the rest.

Correctness rule: source diff narrows the search; exact chunk and configuration hashes make the final reuse decision.

04

Chunk boundaries are the main trap

A small source edit can change many chunks when boundaries depend on absolute positions. The change detector must distinguish a local edit from the downstream boundary effects it causes.

Fixed-size chunking

With 500-character chunks, inserting 20 characters near the start shifts every later 500-character window. Those later chunks genuinely contain different text, so their old embeddings are not safe even though the human edit was small.

Strategy	Effect of insertion	MVP recommendation
Global fixed characters	Many downstream boundaries can shift	Simple baseline; clearly document over-invalidation
Paragraph/heading aware	Later semantic units often retain their boundaries	Preferred when document structure is reliable
Recursive separators	Uses paragraphs, lines and words before hard splitting	Good practical default
Content-defined chunking	Boundaries depend on local content fingerprints	Interesting extension; not required for MVP

Overlapping chunks

If chunk A covers lines 1-25 and chunk B covers lines 20-40, an edit on line 23 affects both chunks. Source spans allow the system to find every overlapping chunk rather than regenerating only one.

Chunk	Stored source span	Edit at lines 23-24	Action
A	1-25	Intersects	Rechunk and verify
B	20-40	Intersects	Rechunk and verify
C	35-55	No intersection	Exact-hash reuse candidate

Be conservative: a false regeneration costs compute; a false reuse can make retrieval incorrect.

Acceptable MVP limitation

Exact-hash matching followed by positional or span-based pairing is sufficient for a semester prototype. Document that major boundary shifts may be reported as additions and deletions rather than perfect human-style modifications.

05

Version the effective pipeline

Identical source text does not guarantee identical chunks or vectors. Every setting that can affect extracted text, normalized text, chunk boundaries or vector values belongs in the reproducibility manifest.

Pipeline stage	Record	Why it matters
Extraction	Library and version, file type, page ordering and extraction options	A different PDF parser can produce different text or reading order
Preprocessing	Ordered operations, parameters and pipeline version	Whitespace, Unicode or case changes alter chunk content
Chunking	Strategy, size, overlap, separators and tokenizer	These settings determine chunk boundaries
Embedding	Provider, model, exact revision, pooling, normalization and dimension	These settings determine vector values and compatibility
Environment	Python and relevant library versions, OS and hardware for benchmarks	Supports reproduction and honest performance comparison
Vector store	Provider, collection schema and sync metadata	Supports rematerialization; storage path alone should not invalidate vectors

Illustrative manifest

```

pipeline:
  extraction: {library: pymupdf, version: 1.x}
  preprocessing:
    steps: [unicode_nfk, normalize_whitespace]
  chunking: {strategy: recursive, size: 500, overlap: 50}
  embedding:
    model: BAAI/bge-small-en-v1.5
    revision:
    normalize: true
  environment:
    python: 3.11
    sentence_transformers:

```

Configuration hash

Serialize the effective configuration in a canonical form with sorted keys, then calculate SHA-256. Exclude settings that cannot affect data, such as CLI color or log verbosity.

Same chunk hash + same effective embedding configuration hash + verified object integrity = safe reuse

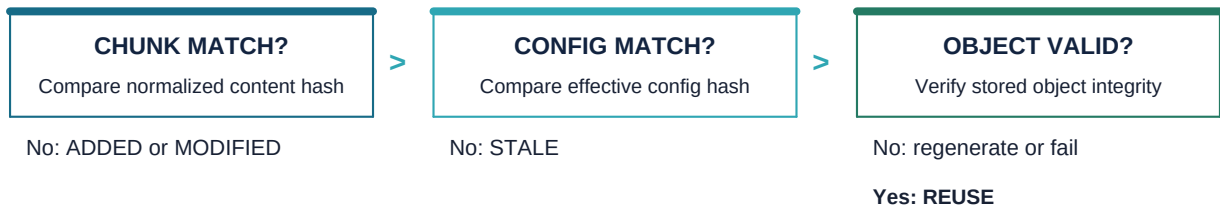
Classification and invalidation rules

Statuses describe what changed and what action is required. They should be derived from source identity, matched chunk content, effective configuration, and object integrity.

Condition	Status	Required action
Matched normalized chunk; same effective configuration	UNCHANGED	Reuse the existing embedding object
Old and current chunks paired by lineage/span/order; content differs	MODIFIED	Generate a new embedding object
Current chunk has no previous match	ADDED	Generate a new embedding object
Previous chunk has no current match	DELETED	Omit from manifest and remove during vector-store sync
Chunk content matches but vector-affecting configuration changed	STALE	Regenerate before creating a valid snapshot
Object is missing or fails hash/integrity verification	CORRUPT/MISSING	Regenerate or fail safely; never silently reuse

Model changes

Equal vector dimensions do not imply compatible semantic spaces. A model or model-revision change should normally invalidate the full collection. Cross-model alignment is optional research and must never be the default correctness mechanism.



The system tracks causes of embedding invalidity. It does not need to compare numeric vectors to decide whether a source edit occurred.

07

What happens during a commit

The system participates before, during and after vector generation. Sentence Transformers performs the model computation; EmbedGit decides when to call it and manages every returned object.

#	Stage	Output
1	Scan	Discover tracked documents and capture stable document identity.
2	Extract + normalize	Run the recorded extraction and preprocessing pipeline.
3	Diff + chunk	Locate source edits, build deterministic chunks and source spans.
4	Classify	Compare with HEAD and label chunks UNCHANGED, MODIFIED, ADDED, DELETED or STALE.
5	Plan	Create a regeneration set and a reuse set.
6	Embed	Call <code>model.encode()</code> only for regeneration-required chunks.
7	Persist	Store immutable vectors and provenance under content-addressed object keys.
8	Commit	Write a manifest referencing documents, chunks, objects, configuration and parent commit.
9	Synchronize	Make the ChromaDB collection match the selected commit.
10	Report	Log generated/reused/deleted counts, duration, failures and benchmark metadata.

Configuration manifest versus execution log

Artifact	Answers	Typical contents
Configuration manifest	How should outputs be produced?	Ordered pipeline, exact parameters, model revision and dependency versions
Execution log	What happened in this run?	Documents scanned, status counts, model calls, duration, errors and hardware
Commit manifest	What exactly belongs to this snapshot?	Parent, message, document/chunk/object references, configuration hash and timestamp

08

Worked example: one small edit

A knowledge base has 100 chunks. A student inserts one paragraph in an operating-systems note. The system should minimize computation without sacrificing correctness.

Before the edit

Collection	Chunks	Embedding objects	Configuration
Commit C1	100	100	bge-small, chunk size 500, overlap 50

During the update

- The document hash changes, so the document is inspected.
- A source diff identifies the inserted paragraph and its surrounding range.
- The affected boundary window is rechunked.
- Exact hash matching recovers 97 unchanged chunks.
- Two paired chunks have changed content; one chunk is completely new.
- Sentence Transformers receives only those three chunks.
- The new commit references 97 old objects and three new objects.

Status	Count	Embedding action
UNCHANGED	97	Reuse existing immutable objects
MODIFIED	2	Generate two new objects
ADDED	1	Generate one new object
DELETED	0	No removal
STALE	0	Configuration unchanged

Result: 3 model calls instead of 100, while the new ChromaDB state still represents all 100 current chunks.

What the benchmark must prove

Compare full regeneration, modified-document regeneration and modified-chunk regeneration on equivalent final data. Report elapsed time, regenerated and reused vector counts, model calls, storage overhead, retrieval quality and reproducibility. Do not claim a speedup until measurements exist.

09

Semester MVP checklist

Build the smallest complete vertical slice first. Advanced alignment, remote collaboration and production deployment are future work, not prerequisites for a successful demonstration.

Milestone	Minimum deliverable	Done when
1. Basic pipeline	TXT/Markdown loading, normalization, deterministic chunking, model call, ChromaDB	A query retrieves relevant chunks
2. Versioning core	.embedgit layout, hashes, status, object store, commit and log	A second commit reuses unchanged objects
3. Diff + restore	Source/chunk comparison, checkout and safe vector-store synchronization	An older commit rematerializes correctly
4. Provenance	Canonical pipeline configuration, hashes, environment and run logs	Every commit explains how it was produced
5. Evaluation	Controlled 1%, 5%, 10% edits and three regeneration baselines	Results include quality, efficiency and limitations
6. Polish	PDF ingestion, tests, CLI output and optional Streamlit UI	Jury demo is repeatable on a clean machine

Questions every team member should be able to answer

- Why is a changed hash not enough to distinguish MODIFIED from ADDED?
- Why must a stable chunk ID never be the only reuse condition?
- How does source diff reduce unnecessary rechunking?
- Why can a small insertion invalidate multiple overlapping chunks?
- Which pipeline settings are part of the configuration hash?
- Why does a model change normally require full regeneration?
- What is canonical history: the commit/object store or the ChromaDB collection?
- How will the benchmark demonstrate savings without overstating results?

Build this first: local, single-user Python CLI + one Sentence Transformers model + ChromaDB + commits, diff, restore and benchmarks.